

Kiểm chứng tính toàn vẹn dữ liệu trong Data Pipeline nhiều giai đoạn bằng Hash-linked Tamper-evident Ledger: một đánh giá bán thực nghiệm

Nguyễn Ngọc Sơn
Học viện Kỹ thuật Mật mã
Email: [CHAT4P016@actvn.edu.vn]

Tóm tắt—Bài báo trình bày một đánh giá bán thực nghiệm đối với cơ chế Hash-linked Tamper-evident Ledger nhằm kiểm chứng tính toàn vẹn dữ liệu trong Data Pipeline nhiều giai đoạn. Cơ chế này ghi nhận bằng chứng toàn vẹn ở từng giai đoạn, liên kết các block ledger bằng `previous_hash`, và sử dụng bộ kiểm chứng để phát hiện sửa đổi sau xử lý. Kết quả thực nghiệm hỗ trợ các giả thuyết chính trong phạm vi giới hạn, đồng thời được diễn giải cùng các mối đe dọa đến tính hợp lệ.

Từ khóa—Tính toàn vẹn dữ liệu, tamper-evident ledger, hash chain, Data Pipeline, đánh giá bán thực nghiệm, an toàn thông tin, Data Lineage.

I. Giới thiệu

Data Pipeline đã trở thành thành phần quan trọng trong các hệ thống thông tin phục vụ vận hành và phân tích dữ liệu. Một pipeline điển hình tiếp nhận dữ liệu nguồn, xử lý qua các tầng trung gian, lưu trữ kết quả đã chuẩn hóa và cung cấp dữ liệu cho báo cáo hoặc ứng dụng hạ nguồn. Trong nhiều kiến trúc lakehouse hoặc warehouse, vòng đời này thường được biểu diễn qua các giai đoạn như Source–Bronze–Silver–Gold [1]. Tuy các giai đoạn này giúp tổ chức dữ liệu rõ ràng hơn, chúng không tự động bảo đảm rằng dữ liệu sẽ không bị thay đổi sau khi pipeline đã hoàn tất. Các nền tảng dữ liệu hiện đại như Delta Lake hỗ trợ nhật ký giao dịch và quản lý dữ liệu theo phiên bản, nhưng bài toán kiểm chứng độc lập sau xử lý vẫn cần một lớp bằng chứng integrity rõ ràng hơn trong ngữ cảnh an toàn thông tin [2, 3].

Đây là vấn đề thuộc phạm vi an toàn thông tin và bảo đảm thông tin. Nếu người dùng có quyền cao, tài khoản bị xâm nhập hoặc người có quyền truy cập nội bộ chỉnh sửa dữ liệu sau khi pipeline chạy xong, công cụ điều phối vẫn có thể hiển thị trạng thái thành công. Cơ chế giám sát thông thường có thể xác nhận rằng tác vụ đã chạy, nhưng không nhất thiết kiểm chứng độc lập rằng dữ liệu lưu

trữ sau đó vẫn còn nguyên vẹn. Nhật ký kiểm toán cơ sở dữ liệu có thể hỗ trợ điều tra, nhưng thường nằm trong cùng ranh giới quản trị với dữ liệu được bảo vệ. Mã kiểm tra ở tầng ứng dụng có thể phát hiện sai lệch đơn giản, nhưng nếu không được liên kết theo chuỗi giữa các giai đoạn thì khó cung cấp bằng chứng truy vết rõ ràng đến transformation bị ảnh hưởng.

Nghiên cứu này là một hướng tiếp cận nhẹ hơn: Hash-linked Tamper-evident Ledger. Cơ chế tamper-evident không ngăn chặn trực tiếp việc chỉnh sửa dữ liệu. Thay vào đó, nó tạo ra bằng chứng để phát hiện rằng việc chỉnh sửa đã xảy ra. Hash-linked Ledger lưu các bản ghi bằng chứng theo thứ tự, trong đó mỗi block chứa hash của block trước đó [4]. Nếu nội dung của block hoặc liên kết `previous_hash` bị thay đổi, quá trình verification về sau có thể phát hiện chuỗi không còn khớp. Cách liên kết bằng hash kế thừa ý tưởng cốt lõi của blockchain [5], nhưng nghiên cứu này chỉ đánh giá một ledger nhẹ, không bao gồm consensus, smart contract hoặc mạng phi tập trung đầy đủ.

A. Khoảng trống nghiên cứu

Một số cơ chế hiện có đã giải quyết từng phần của bài toán toàn vẹn dữ liệu. Nhật ký kiểm toán cơ sở dữ liệu ghi lại thao tác, nhưng quản trị viên có quyền cao có thể tác động đến cả dữ liệu và log [6]. Mã kiểm tra ở tầng ứng dụng có thể xác thực một ảnh chụp trạng thái dữ liệu, nhưng thường bị cô lập và không bảo toàn quan hệ giữa các giai đoạn. Blockchain phi tập trung đầy đủ có thể cung cấp đặc tính bất biến mạnh hơn thông qua consensus và nhiều nút sao chép, nhưng chi phí phát sinh vận hành và độ phức tạp kiến trúc thường vượt quá nhu cầu của một bài toán kiểm chứng integrity nội bộ trong Data Pipeline. Pipeline monitoring quan sát trạng thái

thực thi, nhưng không nhất thiết cung cấp bằng chứng kiểm chứng sau khi pipeline đã hoàn tất.

Khoảng trống mà bài báo này tập trung là thiếu một đánh giá có kiểm soát về cơ chế Hash-linked Ledger nhẹ cho kiểm chứng tính toàn vẹn của Data Pipeline nhiều giai đoạn. Nghiên cứu xem xét liệu cơ chế này có phát hiện được các kịch bản tamper được định nghĩa trước hay không, có xác định được block sai lệch đầu tiên và ngữ cảnh lineage liên quan hay không, và chi phí phát sinh bổ sung thay đổi như thế nào trong một môi trường thực nghiệm bị ràng buộc.

B. Câu hỏi nghiên cứu

Nghiên cứu được dẫn dắt bởi bốn câu hỏi nghiên cứu:

- **RQ1:** Cơ chế Hash-linked Ledger và bộ kiểm chứng có phát hiện được các sửa đổi trái phép đối với dữ liệu pipeline, nội dung ledger và liên kết chuỗi hay không?
- **RQ2:** Cơ chế này có xác định được block sai lệch đầu tiên, giai đoạn bị ảnh hưởng và lineage event liên quan hay không?
- **RQ3:** Cơ chế này tạo thêm chi phí phát sinh thời gian xử lý như thế nào khi số lượng bản ghi thay đổi?
- **RQ4:** Những biến không kiểm soát được trong môi trường thực nghiệm giới hạn sức mạnh kết luận ra sao?

C. Đóng góp

Bài báo có ba đóng góp chính. Thứ nhất, nghiên cứu định nghĩa một cơ chế tamper-evident có phạm vi rõ ràng cho kiểm chứng integrity của Data Pipeline bằng `batch_hash`, `block_hash`, liên kết `previous_hash` và lineage event ở từng giai đoạn. Thứ hai, nghiên cứu thiết kế một đánh giá bán thực nghiệm với điều kiện sạch, điều kiện bị tamper và điều kiện benchmark. Thứ ba, nghiên cứu báo cáo quan sát thực nghiệm cùng các giới hạn validity, thay vì tuyên bố quá mức rằng cơ chế này đạt tính bất biến như Blockchain đầy đủ hoặc đưa ra kết luận nhân quả mạnh.

II. Phương pháp nghiên cứu

A. Chứng minh lựa chọn thiết kế bán thực nghiệm

Nghiên cứu này sử dụng phương pháp bán thực nghiệm (quasi-experimental) [7] vì can thiệp kỹ thuật có thể được triển khai và đo lường, nhưng môi trường thực nghiệm không thể được kiểm soát hoàn toàn. Một true experiment đòi hỏi random assignment, tài nguyên tính toán cố định, điều kiện mạng ổn định, các lần chạy lặp lại trong trạng thái tài nguyên giống hệt nhau và tập đối tượng tấn công đại diện cho thực tế. Những giả định này

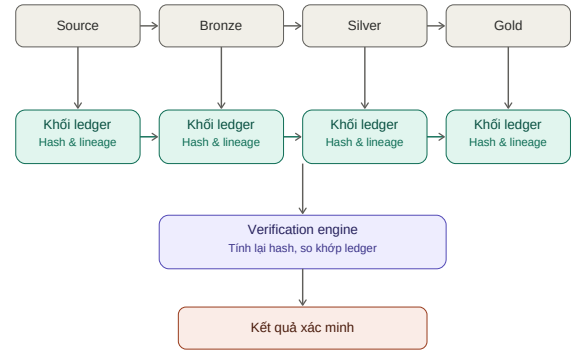


Fig. 1. Hash-linked Tamper-evident Ledger cho Data Pipeline nhiều giai đoạn.

không khả thi trong môi trường Databricks được sử dụng trong nghiên cứu.

Thực nghiệm kiểm soát các yếu tố có thể kiểm soát được: cấu trúc pipeline, seed sinh dữ liệu tổng hợp, danh sách kịch bản tamper, logic kiểm chứng, tập biến đo lường và thứ tự quy trình. Đồng thời, nghiên cứu ghi nhận các yếu tố không thể kiểm soát: lập lịch trên nền tảng đám mây, trạng thái khởi động, cache, độ trễ mạng và việc các kịch bản tamper là đại diện gần đúng (proxy) do nhà nghiên cứu thiết kế, không phải mẫu ngẫu nhiên của không gian tấn công thực tế. Vì vậy, kết quả được diễn giải là bằng chứng bán thực nghiệm về quan hệ giữa điều kiện can thiệp và kết quả quan sát được, không phải kết luận nhân quả phổ quát.

B. Treatment và Control

Treatment là việc bổ sung Hash-linked Tamper-evident Ledger, lineage recorder và bộ kiểm chứng vào một Data Pipeline chuẩn. Điều kiện đối chứng phụ thuộc vào câu hỏi nghiên cứu. Đối với phát hiện và truy vết, trạng thái pipeline sạch, không có kịch bản tamper, đóng vai trò pre-test control. Đối với chi phí phát sinh, pipeline baseline không có ledger và verification instrumentation đóng vai trò non-equivalent control.

C. Giả thuyết nghiên cứu

Tương ứng với bốn câu hỏi nghiên cứu, bốn giả thuyết được phát biểu trong Bảng I.

D. Biến nghiên cứu

Các biến độc lập gồm kịch bản tamper, số lượng bản ghi, giai đoạn pipeline, cơ chế bảo mật và điều kiện chạy. Các biến phụ thuộc gồm kết quả kiểm chứng, tỷ lệ phát hiện, độ chính xác của block sai lệch đầu tiên, mức độ đầy đủ của truy vết, overhead bảo mật, thời gian chạy baseline,

TABLE I
Giả thuyết nghiên cứu

Mã	Giả thuyết
H1	Treatment phát hiện tất cả kịch bản tamper được định nghĩa trước, trong khi điều kiện sạch vẫn hợp lệ.
H2	Treatment xác định được block sai lệch đầu tiên và liên kết được với lineage event tương ứng.
H3	Treatment làm tăng thời gian xử lý so với pipeline baseline, và chi phí phát sinh thay đổi theo số lượng bản ghi.
H4	Các biến không kiểm soát được khiến nghiên cứu không thể đưa ra kết luận nhân quả mạnh như true experiment.

TABLE II
Các biến thực nghiệm cốt lõi

Biến	Loại	Vai trò
Kịch bản tamper	IV phân loại	Tác nhân kiểm thử phát hiện và truy vết.
Số lượng bản ghi	IV thứ bậc	Yếu tố đánh giá mức thay đổi chi phí theo quy mô dữ liệu.
Giai đoạn pipeline	IV phân loại	Ngữ cảnh cho block và ảnh xạ lineage.
Cơ chế bảo mật	IV nhị phân	So sánh control và treatment.
Kết quả kiểm chứng	DV phân loại	Kết quả phát hiện chính.
Tỷ lệ phát hiện	DV số	Tỷ lệ kịch bản tamper được phát hiện.
Overhead thời gian chạy	DV số	Chi phí thời gian bổ sung của treatment.

thời gian chạy có cơ chế bảo vệ và thời gian kiểm chứng. Bảng II tóm tắt các biến cốt lõi cùng loại và vai trò của chúng.

III. Thiết kế và môi trường thực nghiệm

A. Nguyên mẫu Data Pipeline

Nguyên mẫu triển khai một pipeline phân tích gồm bốn giai đoạn: Source, Bronze, Silver và Gold. Dữ liệu nguồn được sinh tổng hợp bằng random seed cố định nhằm hỗ trợ khả năng tái lập. Bronze lưu dữ liệu sau khi nạp dữ liệu, Silver thực hiện chuẩn hóa và transformation, còn Gold lưu kết quả tổng hợp phục vụ phân tích. Mỗi giai đoạn sinh ra bằng chứng integrity và ghi vào ledger.

Đối với mỗi giai đoạn, ledger ghi nhận tên giai đoạn, số lượng bản ghi, schema_hash, batch_hash, block_hash, previous_hash, metadata của transformation, dấu thời gian và định danh lần chạy pipeline. Trường batch_hash đại diện cho nội dung đã chuẩn hóa chính xác của output tại giai đoạn. Trường schema_hash đại diện cho cấu trúc dữ liệu. Trường block_hash là giá trị băm (digest) mật mã của nội dung trong block. Trường previous_hash

Fig. 2. Môi trường triển khai thực nghiệm trên Databricks Free Edition, gồm các notebook từ 00_setup_environment đến 07_experiment_and_metrics trong thư mục notebooks, đồng bộ từ repository Git và thực thi tuần tự luồng Source-Bronze-Silver-Gold.

TABLE III
Kịch bản tamper và kết quả kiểm chứng kỳ vọng

Kịch bản	Kết quả kỳ vọng
NONE	VALID
MODIFY_TRANSACTION_AMOUNT	DATA_TAMPERED
DELETE_TRANSACTION	RECORD_COUNT_MISMATCH
INSERT_FAKE_TRANSACTION	RECORD_COUNT_MISMATCH
MODIFY_LEDGER_BATCH_HASH	DATA_TAMPERED
MODIFY_LEDGER_TRANSFORMATION	BLOCK_TAMPERED
MODIFY_LEDGER_PREVIOUS_HASH	CHAIN_BROKEN

liên kết block hiện tại với block trước đó trong cùng lần chạy pipeline.

Bộ kiểm chứng tính toán lại bằng chứng ở từng giai đoạn và so sánh với ledger đã lưu. Sai lệch mã băm nội dung cho thấy khả năng dữ liệu đã bị sửa đổi không hợp lệ. Sai lệch số lượng bản ghi cho thấy khả năng chèn hoặc xóa. Sai lệch nội dung block cho thấy khả năng ledger đã bị sửa đổi không hợp lệ. Sai lệch previous_hash cho thấy liên kết chuỗi bị phá vỡ.

B. Kịch bản tamper

Nghiên cứu đánh giá sáu kịch bản tamper và một kịch bản sạch. Kịch bản sạch, ký hiệu NONE, được dùng để quan sát dương tính giả. Các kịch bản tamper bao gồm sửa giá trị giao dịch, xóa bản ghi, chèn bản ghi giả, sửa batch_hash đã lưu, sửa metadata của transformation và sửa liên kết previous_hash. Bảng III liệt kê từng kịch bản cùng kết quả kiểm chứng kỳ vọng.

C. Quy trình thực nghiệm

Quy trình thực nghiệm được mô tả trong Hình 3. Đầu tiên, môi trường và các Delta tables được khởi tạo. Tiếp theo, dữ liệu tổng hợp được sinh bằng seed cố định. Sau đó, pipeline baseline và pipeline có treatment được thực thi. Bộ kiểm chứng kiểm tra clean run. Một kịch bản tamper được áp dụng, rồi verification được chạy lại để đo phát hiện và truy vết. Trạng thái baseline được reset trước khi

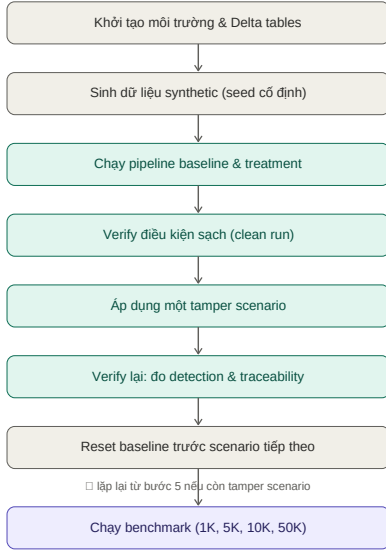


Fig. 3. Quy trình thực nghiệm cho clean run, tamper run và benchmark run.

chuyển sang scenario tiếp theo. Cuối cùng, benchmark được chạy theo nhiều mức record count.

Các lần chạy khởi động (warm-up) được đánh dấu và loại khỏi phần tổng hợp benchmark chính. Median chi phí phát sinh được ưu tiên hơn mean vì trạng thái cloud compute và khởi động nguội (cold start) có thể tạo giá trị ngoại lai. Benchmark sử dụng các mức số lượng bản ghi 1K, 5K, 10K và 50K.

IV. Đánh giá thực nghiệm

A. Kết quả phát hiện tamper

Điều kiện sạch tạo ra kết quả kiểm chứng VALID trong dashboard được xuất từ Databricks SQL, được dùng làm bằng chứng cho trạng thái chưa bị tamper. Các thực nghiệm can thiệp tạo ra kết quả khác VALID đối với tất cả kịch bản tamper được định nghĩa trước. Bảng IV tóm tắt các kết quả quan sát được. Kết quả hỗ trợ H1 trong phạm vi tập kịch bản đã được định nghĩa và đánh giá.

Tỷ lệ phát hiện được tính theo công thức:

$$\text{Tỷ lệ phát hiện} = \frac{TP}{TP + FN} \times 100\%. \quad (1)$$

Với sáu trường hợp dương tính thật và không quan sát thấy âm tính giả trong các kịch bản có can thiệp, tỷ lệ phát hiện đạt 100% trong tập kiểm thử đã định nghĩa trước. Kết quả này không nên được hiểu là mức bao phủ phổ quát đối với mọi vector tấn công. Nó chỉ có nghĩa là cơ chế đã phát hiện sáu vector can thiệp được lựa chọn trong thực nghiệm này.

TABLE IV
Kết quả kiểm chứng quan sát được

Kịch bản	Kết quả quan sát	Loại
NONE	VALID	TN
MODIFY TRANSACTION_AMOUNT	DATA_TAMPERED	TP
DELETE TRANSACTION	RECORD_COUNT_MISMATCH	TP
INSERT FAKE TRANSACTION	RECORD_COUNT_MISMATCH	TP
MODIFY LEDGER_BATCH_HASH	DATA_TAMPERED	TP
MODIFY LEDGER TRANSFORMATION	BLOCK_TAMPERED	TP
MODIFY LEDGER PREVIOUS_HASH	CHAIN_BROKEN	TP

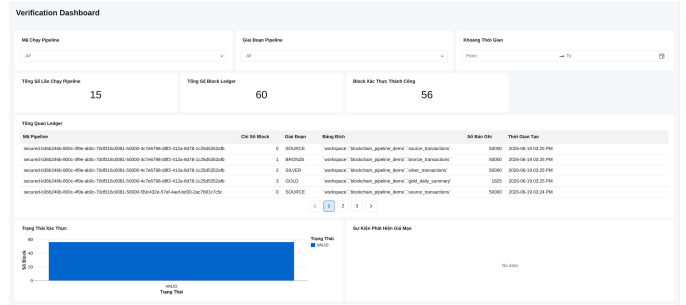


Fig. 4. Verification Dashboard ở trạng thái sạch: dashboard ghi nhận các lần chạy pipeline, số block ledger, số block xác thực thành công và kết quả VALID trong bộ lọc hiện tại.

B. Kết quả truy vết

Bộ kiểm chứng trả về block sai lệch đầu tiên khi phát hiện sai khớp. Block này có thể được liên kết với các sự kiện lineage [8] thông qua định danh lần chạy pipeline. Luồng lineage mẫu ghi nhận các transformation Source-Bronze, Bronze-Silver và Silver-Gold, bao gồm số lượng bản ghi đầu vào, số lượng bản ghi đầu ra, ngưỡng transformation và trạng thái. Đối với các kịch bản tamper được đánh giá, block sai lệch đầu tiên và ngưỡng lineage khớp với giai đoạn bị ảnh hưởng ở mức giai đoạn tổng quát. Điều này hỗ trợ H2, nhưng vẫn có giới hạn: một số giá trị giai đoạn trong báo cáo nguồn được ghi nhận ở mức xấp xỉ và cần được xác minh bằng SQL xuất dữ liệu trực tiếp để có kết luận mạnh hơn. Panel “Block Bị Phá Vỡ Đầu Tiên” trong dashboard chưa hiển thị dữ liệu trong lần xuất này; phép JOIN chi tiết giữa block sai lệch đầu tiên và sự kiện lineage cần được thực hiện qua KPI 6 (sql/dashboard_queries.sql) để đối chiếu đầy đủ.

C. Kết quả overhead thời gian chạy

Overhead thời gian chạy được tính theo công thức, với T_{secured} và T_{baseline} lần lượt là thời gian xử lý pipeline có và không có Hash-linked Ledger:

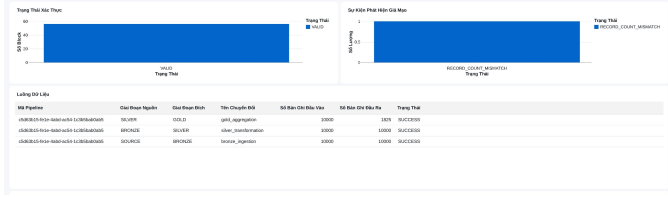


Fig. 5. Verification Dashboard sau khi áp dụng một kịch bản tamper. Dashboard ghi nhận kết quả RECORD_COUNT_MISMATCH và hiển thị các sự kiện lineage Source→Bronze→Silver→Gold.

TABLE V
Overhead thời gian chạy xấp xỉ theo số lượng bản ghi

Số bản ghi	Overhead xấp xỉ	Bằng chứng
1K	~160%	Dashboard
5K	~155-160%	Dashboard
10K	~180%	Dashboard
50K	~220%	Dashboard

$$\text{Overhead thời gian chạy} = \frac{T_{\text{secured}} - T_{\text{baseline}}}{T_{\text{baseline}}} \times 100\% \quad (2)$$

Các giá trị xấp xỉ từ dashboard cho thấy overhead tăng từ khoảng 160% ở mức 1K bản ghi lên khoảng 220% ở mức 50K bản ghi. Bảng V tóm tắt xu hướng quan sát được. Trường hợp 5K gần với 1K, cho thấy khả năng có ảnh hưởng của dao động thời gian chạy trên môi trường cloud. Nhìn chung, bằng chứng thực nghiệm hỗ trợ H3, nhưng kết luận cần được diễn giải thận trọng do còn tồn tại các yếu tố gây nhiễu trong môi trường cloud.

Độ trễ kiểm chứng dao động xấp xỉ từ 24 giây đến 31 giây trong các lần chạy gần nhất, với một giá trị ngoại lai cao hơn. Quan sát này phù hợp với giả định bán thực nghiệm rằng khởi động nguội (cold start) và cơ chế lập lịch trên môi trường cloud không thể được kiểm soát hoàn toàn.

D. Thảo luận

Bằng chứng thực nghiệm cho thấy Hash-linked Ledger có thể làm cho hành vi sửa đổi sau xử lý trở nên quan sát được trong pipeline nhiều giai đoạn. Cơ chế này đặc biệt hữu ích khi mục tiêu không phải là ngăn chặn trực tiếp các thao tác ghi bởi tài khoản đặc quyền, mà là phát hiện các sửa đổi trái phép trong quá trình kiểm chứng về sau. Điều này phù hợp với thiết kế tamper-evident, tức là phát hiện sửa đổi sau khi xảy ra, chứ không phải tamper-proof, tức là ngăn chặn tuyệt đối mọi sửa đổi.

Kết quả cũng làm rõ vai trò của thuật ngữ Blockchain. Nguyên mẫu sử dụng hash chain lấy cảm hứng từ Blockchain, nhưng không triển khai đồng thuận phân tán, các nút sao chép, mining, proof-of-work, proof-of-stake hoặc smart contract.

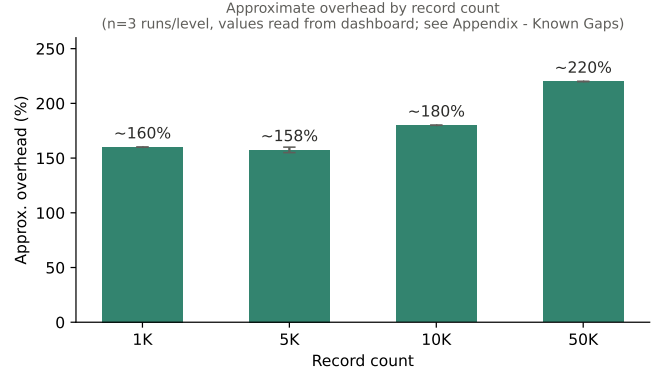


Fig. 6. Overhead thời gian chạy xấp xỉ theo số lượng bản ghi, đọc từ dashboard với n=3 lần chạy cho mỗi mức record count.

Observed verification latency range (from dashboard, no per-run raw export)
- illustrative min / typical / outlier, NOT a full box-plot (n<10)

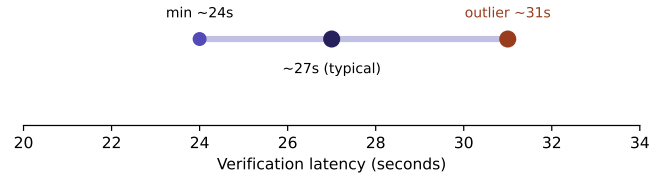


Fig. 7. Khoảng độ trễ kiểm chứng quan sát được từ dashboard. Hình chỉ minh họa giá trị nhỏ nhất, điển hình và ngoại lai; chưa phải box-plot đầy đủ do chưa có dữ liệu thô cho từng lần chạy.

Vì vậy, thuật ngữ chính xác hơn là Hash-linked Tamper-evident Ledger. Sự phân biệt này quan trọng đối với tính hợp lệ khái niệm và giúp tránh tuyên bố quá mức.

V. Các mối đe dọa đến tính hợp lệ

A. Tính hợp lệ nội tại

Mối đe dọa chính đến tính hợp lệ nội tại là trạng thái compute không được kiểm soát hoàn toàn. Thực nghiệm chạy trong môi trường Databricks do cloud quản lý, trong đó cluster khởi động, trạng thái cache, cơ chế lập lịch và phân bổ tài nguyên không cố định. Các yếu tố này có thể ảnh hưởng đến overhead thời gian chạy độc lập với treatment. Để giảm rủi ro này, các lần chạy khởi động (warm-up) được loại khỏi phân tích chính và median được ưu tiên hơn mean.

Một mối đe dọa khác là hiệu ứng lịch sử. Nếu các bảng không được reset đúng giữa các kịch bản tamper, dữ liệu cũ hoặc bản ghi ledger cũ có thể tạo dương tính giả hoặc âm tính giả. Vì vậy, quy trình bao gồm bước RESET_BASELINE bắt buộc trước khi chuyển sang kịch bản tiếp theo.

B. Tính hợp lệ ngoại tại

Thực nghiệm sử dụng một workspace Databricks và dữ liệu giao dịch tổng hợp. Kết quả có thể không khái quát trực tiếp sang AWS Glue, Spark tại chỗ, dbt, Snowflake hoặc môi trường lakehouse production. Dữ liệu tổng hợp cũng thiếu các đặc điểm như phân phối lệch, trôi cấu trúc dữ liệu, giá trị thiếu và độ phức tạp vận hành của dữ liệu thực tế. Các nghiên cứu lặp lại trong tương lai nên triển khai trên nhiều môi trường và sử dụng dữ liệu giống môi trường vận hành thực tế đã ẩn danh.

C. Tính hợp lệ khái niệm

Hệ thống không nên được diễn giải là Blockchain đầy đủ. Khái niệm được đánh giá trong nghiên cứu là Hash-linked Tamper-evident Ledger. Cơ chế này cung cấp bằng chứng toàn vẹn thông qua các giá trị hash và liên kết `previous_hash`, nhưng không cung cấp đồng thuận phân tán hay tính bất biến của public chain. Một mối đe dọa khái niệm khác là chỉ số tỷ lệ phát hiện. Tỷ lệ phát hiện 100% chỉ áp dụng cho sáu kịch bản tamper được lựa chọn trong thực nghiệm.

D. Tính hợp lệ kết luận

Số lượng kịch bản tamper, các mức số lượng bản ghi và số lần lặp lại đo hiệu năng còn hạn chế. Điều này làm cho kết quả hữu ích như bằng chứng bán thực nghiệm, nhưng chưa đủ cho kết luận thống kê mạnh. Các giá trị overhead mang tính xấp xỉ vì được đọc từ dashboard quan sát thay vì xuất trực tiếp từ dữ liệu benchmark thô. Cần ít nhất khoảng 30 lần chạy lặp lại cho mỗi mức số lượng bản ghi để tính khoảng tin cậy, biểu đồ phân phối và phân tích cỡ hiệu ứng mạnh hơn.

VI. Kết luận và hướng phát triển

Bài báo đã trình bày một đánh giá bán thực nghiệm đối với cơ chế Hash-linked Tamper-evident Ledger nhằm kiểm chứng tính toàn vẹn dữ liệu trong hệ thống Data Pipeline. Cơ chế này ghi nhận bằng chứng toàn vẹn ở từng giai đoạn và liên kết các block ledger thông qua `previous_hash`. Bộ kiểm chứng tính toán lại số lượng bản ghi, `schema_hash`, `batch_hash`, `block_hash` và liên kết chuỗi để phát hiện sửa đổi sau xử lý.

Các quan sát thực nghiệm hỗ trợ các giả thuyết chính trong phạm vi giới hạn của nghiên cứu. Tất cả kịch bản tamper được định nghĩa trước đều được phát hiện, điều kiện sạch vẫn VALID trong dashboard được xuất từ Databricks SQL quan sát được, và ngữ cảnh lineage hỗ trợ xác định giai đoạn bị ảnh hưởng. Overhead thời gian chạy tăng khi số lượng bản ghi tăng, nhưng các giá trị

overhead hiện tại vẫn mang tính xấp xỉ và chịu ảnh hưởng bởi dao động của môi trường cloud.

Nghiên cứu nên được hiểu là bằng chứng cho một cơ chế tamper-evident nhẹ, không phải bằng chứng về bảo mật Blockchain đầy đủ. Nguyên mẫu không bao gồm đồng thuận phân tán, nhiều nút độc lập, smart contract hoặc điểm neo tin cậy bên ngoài. Vì vậy, ranh giới bảo mật của cơ chế hiện tại bị giới hạn trong phạm vi workspace thực nghiệm được kiểm soát.

Trong tương lai, nghiên cứu cần được lặp lại trên nhiều nền tảng dữ liệu, mở rộng tập vector can thiệp thông qua kiểm thử đối kháng, tăng số lần lặp lại đo hiệu năng, xuất dữ liệu benchmark thô, đưa ledger sang một ranh giới tin cậy độc lập và bổ sung chữ ký số nhằm tăng cường khả năng phát hiện sửa đổi bởi người dùng nội bộ có đặc quyền.

Appendix A

Phụ lục tái lập kết quả

Phụ lục này cung cấp đủ thông tin để tái lập lại thực nghiệm trong khoảng 15 phút trên một workspace Databricks mới, theo đúng tinh thần minh bạch và trung thực khoa học: các cấu hình chưa ghi nhận được liệt kê rõ là *chưa có bằng chứng* thay vì suy đoán.

A. Mã nguồn và môi trường

- **Repository:** https://github.com/sonnnntech/kma_hk2_chuyen_de_co_so_khoa_hoc
- **Nền tảng:** Databricks Free Edition; notebook Python compute hoặc serverless compute cho notebooks/00--07; Databricks SQL Warehouse chỉ dùng cho dashboard.
- **Phụ thuộc cục bộ (kiểm thử):**
`requirements.txt` — `pyspark>=3.5,<4.0`,
`pytest>=8,<9` (Databricks Runtime đã có sẵn PySpark/Delta Lake).

B. Cấu hình môi trường thực nghiệm

TABLE VI
Cấu hình môi trường (CONFIG/DEMO_CONFIG.PY)

Thành phần	Giá trị / Nguồn
Catalog	workspace
Schema	blockchain_pipeline_demo
Số bản ghi mặc định	10 000
Random seed	42
Source system	SCHOOL_DEMO
Benchmark record counts	1 000 / 5 000 / 10 000 / 50 000
Số lần chạy cho mỗi mức dữ liệu	3 (+ warm-up, loại khỏi tổng hợp)
CPU/RAM của môi trường Databricks	Chưa có bằng chứng ghi nhận — Databricks Free Edition / serverless compute không công bố cấu hình cố định.
Databricks Runtime version	Chưa có bằng chứng ghi nhận — cần ghi lại từ workspace khi chạy lại.

C. Các bước tái lập (~15 phút)

- 1) **Import repo:** Databricks Workspace → Repos → Add Repo → dán URL GitHub ở trên, chọn branch main.
- 2) **Chạy sạch (clean run), tuần tự bằng Python compute:**
notebooks/00_setup_environment.py
notebooks/01_generate_source_data.py
notebooks/02_bronze_ingestion.py
notebooks/03_silver_transformation.py
notebooks/04_gold_aggregation.py
notebooks/05_verify_integrity.py
Kết quả kỳ vọng: SOURCE/BRONZE/SILVER/GOLD = VALID.
- 3) **Chạy một kịch bản tamper:**
notebooks/06_run_tamper_scenarios.py
với widget SCENARIO = MODIFY_TRANSACTION_AMOUNT, CONFIRM_TAMPER = YES. Reset bằng SCENARIO = RESET_BASELINE trước khi đổi scenario khác.
- 4) **Chạy benchmark:**
notebooks/07_experiment_and_metrics.py
với RECORD_COUNTS=1000,5000,10000,50000, RUNS_PER_SIZE=3, INCLUDE_WARMUP=YES.
- 5) **Xuất số liệu chính xác (khuyến nghị):** chạy KPI 8/9 trong sql/dashboard_queries.sql trên experiment_metrics (lọc is_warmup=false) để thay các giá trị xấp xỉ trong Bảng V bằng số liệu thô.
- 6) **Kiểm thử đơn vị cục bộ (tuỳ chọn):**
pip install -r requirements.txt && pytest tests/.

D. Các khoảng trống về khả năng tái lập

Theo đúng yêu cầu trung thực khoa học, các giới hạn sau được công khai thay vì che giấu hoặc làm tròn số liệu:

TABLE VII
Các khoảng trống về khả năng tái lập

Gap	Tác động
CPU/RAM và Databricks Runtime version chưa được ghi nhận	Giảm khả năng tái lập chính xác về hiệu năng; cần bổ sung khi chạy lại.
Bảng V và Fig. 6 là giá trị xấp xỉ đọc từ dashboard chart, chưa export SQL trực tiếp	Cần chạy KPI 8/9 để lấy avg_overhead_percent và median_overhead_percent chính xác.
RUNS_PER_SIZE = 3	Chưa đủ mẫu cho box-plot/CDF đáng tin cậy; Fig. 7 chỉ là minh họa min/điểm hình/outlier, không phải phân phối đầy đủ.
first broken block trong Bảng IV ở mức giai đoạn tổng quát	Cần export JOIN verification_results × lineage_events để xác nhận chi tiết.

References

- [1] Databricks. (2024) What is a medallion architecture? Truy cập: 2026/07/05. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>
- [2] —. (2024) Delta lake: Reliable data lakes at scale. Truy cập: 2026/07/05. [Online]. Available: <https://www.databricks.com/product/delta-lake-on-databricks>
- [3] International Organization for Standardization, *ISO/IEC 27001: Information security, cybersecurity and privacy protection*, ISO/IEC Std., 2022.
- [4] R. C. Merkle, “Secure communications over insecure channels,” *Communications of the ACM*, vol. 21, no. 4, pp. 294–299, 1978.
- [5] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [6] B. Schneier and J. Kelsey, “Cryptographic support for secure logs on untrusted machines,” in *Proceedings of the 7th USENIX Security Symposium*, San Antonio, TX, 1998, pp. 53–62.
- [7] W. R. Shadish, T. D. Cook, and D. T. Campbell, *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*. Boston, MA: Houghton Mifflin, 2002.
- [8] OpenLineage Project (Linux Foundation). (2024) Openlineage: An open standard for metadata and lineage collection. Truy cập: 2026/07/05. [Online]. Available: <https://openlineage.io>